Helm e ArgoCD

Guia simples e didático para iniciantes

Empacotando e implantando aplicações no Kubernetes com GitOps



Ada • DE-OP-011 Orquestração • Kubernetes | Aula 09

Objetivos de hoje

- ✓ Entender por que gerenciar YAML “na mão” não escala
- ✓ Conhecer o Helm como gerenciador de pacotes do Kubernetes
- ✓ Dominar os conceitos de Chart, Templates/Values e Releases
- ✓ Compreender o que é GitOps e o papel do ArgoCD
- ✓ Ver como o ArgoCD sincroniza o cluster com o repositorio Git
- ✓ Entender como Helm e ArgoCD se complementam no fluxo de deploy

O problema: YAML “na mão” não escala

Uma aplicação no Kubernetes precisa de vários arquivos YAML (Deployment, Service, ConfigMap, Ingress...).

Duplicidade

Os mesmos valores repetidos em dezenas de arquivos separados.

Erros manuais

Editar YAML à mão em cada ambiente é propenso a falhas.

Sem histórico

Difícil saber o que mudou e voltar atrás quando algo quebra.

Configuration Drift

O cluster “ao vivo” fica diferente do que está versionado.

Fluxo de trabalho integrado

Do empacotamento à aplicação rodando no cluster — em quatro etapas:



1. O que é o Helm?

O gerenciador de pacotes do Kubernetes

Pense no Helm como uma **“App Store”** do Kubernetes.

Ele funciona como o **apt** no Linux, o **pip** no Python ou o **npm** no Node.js.

Em vez de gerenciar dezenas de arquivos soltos, o Helm agrupa tudo em um único pacote: o **Helm Chart**.

💡 Analogia

O Helm é como uma **receita de bolo adaptável**.

A estrutura já está pronta; você muda apenas a quantidade de açúcar ou o sabor do recheio no arquivo de valores — sem reescrever o modo de preparo.

Conceitos-chave do Helm

1

Helm Chart

Uma coleção de arquivos de configuração pré-montados do Kubernetes. Use charts prontos da comunidade ou crie os seus.

2

Templates & Values

Em vez de dados fixos, o Helm usa variáveis. Você ajusta senhas, portas ou memória no arquivo values.yaml sem mexer no código principal.

3

Releases

Cada instalação ou alteração gera um histórico de versões. Se algo der errado, um único comando faz o rollback para a versão anterior.

2. O que é o ArgoCD?

A automação de implantação com GitOps

Enquanto o Helm **empacota** a aplicação, o ArgoCD **automatiza** a forma como ela é publicada no cluster.

GitOps é usar um repositório Git como a **única fonte da verdade**. Mudou o Git → o cluster é atualizado automaticamente.

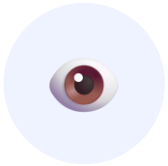
Isso elimina o Configuration Drift: o cluster nunca fica diferente do que está versionado.

💡 Analogia

O ArgoCD é como o **piloto automático** de um avião.

Ele confere o mapa planejado (Git) e ajusta o manche (Kubernetes) continuamente, garantindo que a rota real seja igual à planejada.

Como o ArgoCD funciona



1. Monitora o Git

Fica vigiando o repositório continuamente, observando o estado desejado da aplicação.



2. Detecta diferenças

Ao mudar um arquivo no Git, percebe que o cluster está desatualizado (Out of Sync).

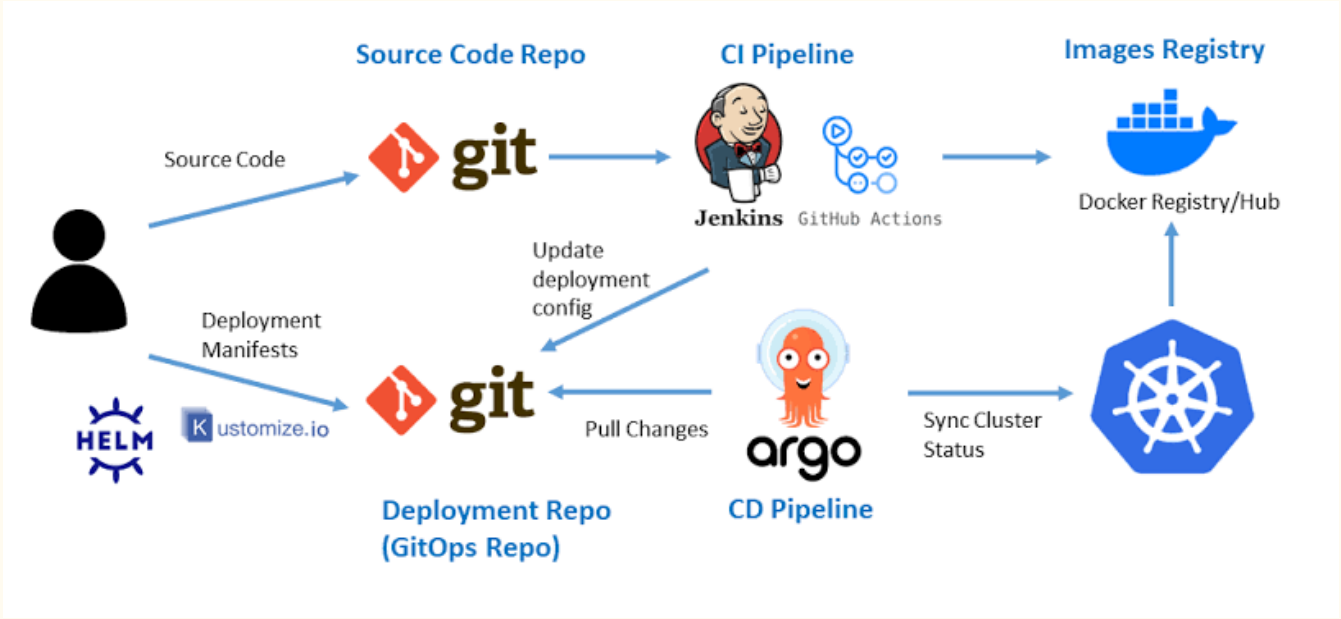


3. Sincroniza sozinho

Puxa a nova versão do Git e atualiza o Kubernetes sem intervenção manual.

3. Como Helm e ArgoCD trabalham juntos

O Helm organiza e cria o pacote flexível; o **ArgoCD** lê esse pacote no Git e mantém o Kubernetes sempre atualizado.



Fonte da imagem: fluxo GitOps — Source Code → CI → Registry / Deployment Repo → ArgoCD → Kubernetes

Helm x ArgoCD: papéis que se completam

	Helm	ArgoCD
Objetivo principal	Gerenciar e parametrizar pacotes da aplicação	Implantação contínua e sincronização GitOps
Analogia didática	O livro de receitas empacotado	O robô cozinheiro que segue a receita
Atua principalmente	Antes do deploy (empacotamento)	Durante e após o deploy (sincronização contínua)

MÃOS À OBRA

Exemplo prático

Vamos estruturar um repositório GitOps de verdade:

- o **Chart do Helm** (a receita da aplicação)
- o **Application do ArgoCD** (o piloto automático)







1. Organizando a estrutura no Git

Boa prática GitOps: separe os **Charts do Helm** das **configurações do ArgoCD**.

estrutura do repositório

```
meu-projeto-gitops/
│
├── helm/                        # A "receita" da aplicação
│   └── minha-app/
│       ├── Chart.yaml           # Metadados do Chart (nome, versão)
│       ├── values.yaml          # Valores padrão da aplicação
│       └── templates/           # Modelos de arquivos Kubernetes
│           ├── deployment.yaml  # Como a aplicação vai rodar
│           └── service.yaml     # Como é exposta na rede
│
└── argocd/                      # O "piloto automático"
    └── application.yaml         # O que o ArgoCD deve sincronizar
```

2. Arquivos do Helm — Chart & Values

O Helm junta os **templates** com as variáveis do **values.yaml** para gerar o YAML final.

Chart.yaml

```yaml
apiVersion: v2
name: minha-app
description: Chart Helm de exemplo
type: application
version: 1.0.0        # Versão do pacote
appVersion: "2.0.0"   # Versão da app
```

A certidão de nascimento do pacote

values.yaml

```yaml
replicaCount: 2       # cópias (pods)

image:
  repository: nginx
  tag: "1.25"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80
```

As variáveis que os alunos podem alterar

2. Arquivos do Helm — Templates

Os templates usam as variáveis via `{{ .Values ... }}` — nada é fixo no código.

templates/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deployment
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.port }}
```

templates/service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-service
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.port }}
  selector:
    app: {{ .Release.Name }}
```

3. Arquivo do ArgoCD — a mágica do GitOps



O **Application** diz ao ArgoCD qual repositório vigiar.

argocd/application.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: minha-app-argocd
  namespace: argocd
spec:
  project: default
  source:                # 1. FONTE (Git)
    repoURL: 'https://github.com/voce/gitops.git'
    targetRevision: HEAD
    path: 'helm/minha-app'
    helm:
      valueFiles: [values.yaml]
  destination:           # 2. DESTINO (K8s)
    server: 'https://kubernetes.default.svc'
    namespace: producao
  syncPolicy:            # 3. AUTOMAÇÃO
    automated:
      prune: true        # apaga o que sai do Git
      selfHeal: true     # corrige alterações manuais
```

1 FONTE

Onde está o código no Git: URL, branch e o caminho do Chart.

2 DESTINO

Em qual cluster e namespace instalar a aplicação.

3 AUTOMAÇÃO

prune + selfHeal = o cluster segue o Git, sempre.

ada

Dica para a aula: veja o GitOps em ação

Uma demonstração de 30 segundos que fixa todo o conceito:

1

Helm

Altere
replicaCount: 2 → 5
no values.yaml

2

Git

git commit
+ git push
para o GitHub

3

ArgoCD

Detecta a mudança e
cria os novos pods
automaticamente

Resumo da aula

Helm

Empacota a aplicação em Charts reutilizáveis e parametrizáveis, com histórico de versões e rollback.

GitOps

Usa o Git como única fonte da verdade — o estado desejado do cluster vive versionado no repositório.

ArgoCD

Sincroniza o cluster com o Git de forma contínua e automática, eliminando o Configuration Drift.

Juntos

Helm cria o pacote flexível; ArgoCD garante que o Kubernetes reflita sempre o que está no Git.

Perguntas?

Helm empacota • GitOps versiona • ArgoCD sincroniza

